



SVEUČILIŠTE U ZAGREBU



Fakultet
elektrotehnike i
računarstva

Sveučilišni prijediplomski studij

Elektrotehnika i informacijska
tehnologija

Računarstvo

Komunikacijske mreže 2

13. Obrada toka podataka

Ak. god. 2025./2026.

Creative Commons



- slobodno smijete:

- **dijeliti** — umnožavati, distribuirati i javnosti priopćavati djelo
- **prerađivati** djelo



- pod sljedećim uvjetima:

- **imenovanje:** morate priznati i označiti autorstvo djela na način kako je specificirao autor ili davatelj licence (ali ne način koji bi sugerirao da Vi ili Vaše korištenje njegova djela imate njegovu izravnu podršku).
- **nekomercijalno:** ovo djelo ne smijete koristiti u komercijalne svrhe.
- **dijeli pod istim uvjetima:** ako ovo djelo izmijenite, preoblikujete ili stvarate koristeći ga, preradu možete distribuirati samo pod licencom koja je ista ili slična ovoj.



U slučaju daljnjeg korištenja ili distribuiranja morate drugima jasno dati do znanja licencne uvjete ovog djela.

Od svakog od gornjih uvjeta moguće je odstupiti, ako dobijete dopuštenje nositelja autorskog prava.

Ništa u ovoj licenci ne narušava ili ograničava autorova moralna prava.

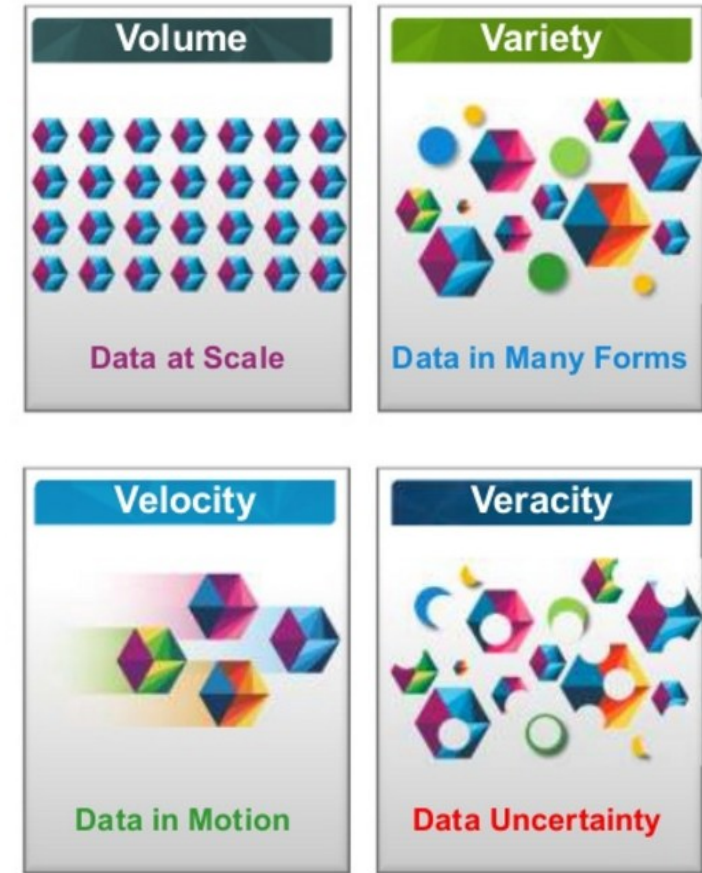
Tekst licence preuzet je s <http://creativecommons.org/>

Definicija velikih podataka (*Big Data*)

- “*Big Data - the massive amounts of data collected over time that are difficult to analyze and handle using common database management tools. The data are analyzed for marketing trends in business as well as in the fields of manufacturing, medicine and science. The types of data include **business transactions, e-mail messages, photos, surveillance videos, activity logs** and **unstructured text from blogs** and **social media**, as well as **the huge amounts of data** that can be collected **from sensors** of all varieties.*” [1]
- „*Big data analytics is the process of **examining large amounts of data** of a variety of types **to uncover hidden patterns, unknown correlations and other useful information.***” [2]

Obilježja velikih podataka

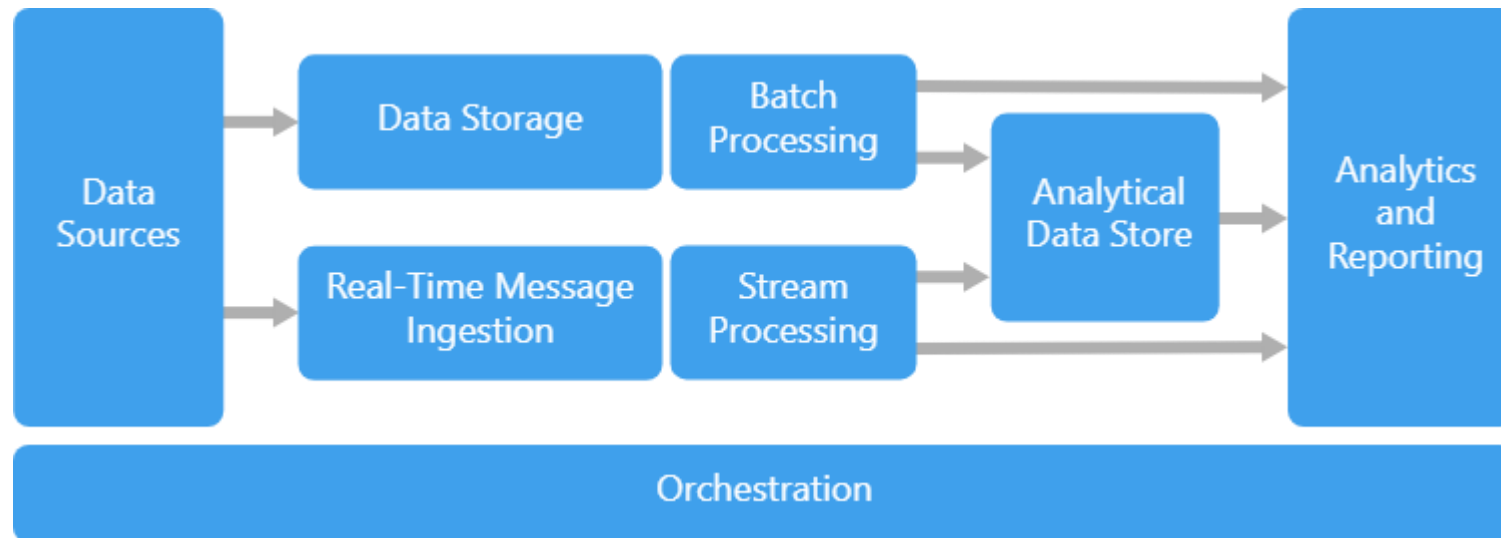
- Veliki podaci su oni koji su **preveliki, prebrzi, prerasnoliki ili s prerasličitom razinom vjerodostojnosti** da bi se mogli pohranjivati i učinkovito obrađivali korištenjem tradicionalnih baza podataka
- Obilježja velikih podataka (*four V's*)
 - Volumen (*Volume*)
 - Raznolikost (*Variety*)
 - **Brzina (*Velocity*)**
 - Vjerodostojnost (*Veracity*)



Izvor: <http://www.slideshare.net/findwise/ibm-big-dataanalytics>

Obrada velikih podataka

- Veliki podaci se obrađuju specijaliziranim platformama
- Platforma za obradu velikih podataka
 - integrirani sustav koji omogućuje prikupljanje, pohranu, obradu i analizu velikih količina podataka
 - obrada **tokova (dinamičnih) podataka** u stvarnom vremenu (*streaming processing*) ili statičnih podataka u serijama (*batch processing*)



Izvor: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/big-data>

Najpopularnije platforme za obradu velikih podataka



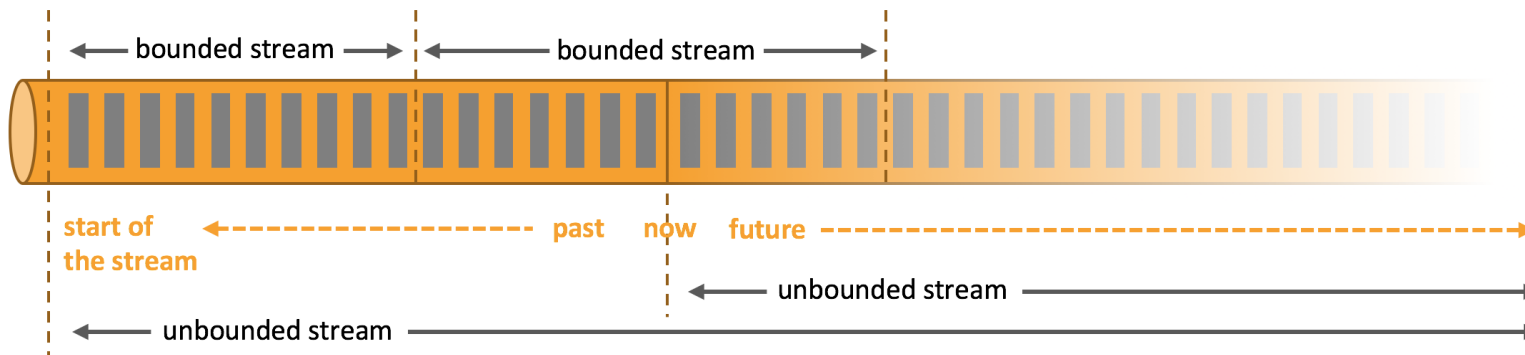
Izvor: <https://www.linkedin.com/pulse/100-open-source-big-data-ml-architecture-papers-sequel-anil-madan/>

Značajke platformi za obradu velikih podataka

- Skalabilnost
 - sposobnost prilagodbe sustava na povećanje ili smanjenje broja zahtjeva (u jedinici vremena)
- Raspodijeljenost
 - podaci su često raspoređeni na više čvorova te se obrađuju u paraleli radi veće učinkovitosti
- Otpornost na neispravnosti
 - visoka dostupnost i pouzdanost čak i u slučaju kvarova i ispada čvorova

Tok podataka (*data stream*)

- “A data stream is **a real-time, continuous, ordered** (implicitly by arrival time or explicitly by timestamp) **sequence of items**. It is **impossible to control the order** in which items arrive, **nor** is it feasible **to locally store a stream in its entirety**.” [3]
- Potencijalno beskonačan slijed podatakovnih objekata
- Mora se odmah početi obrađivati umjesto da se čeka da se prikupi veća količina podataka za obradu
- Kao znanstveni problem se javljaju još 70-ih godina prošlog stoljeća

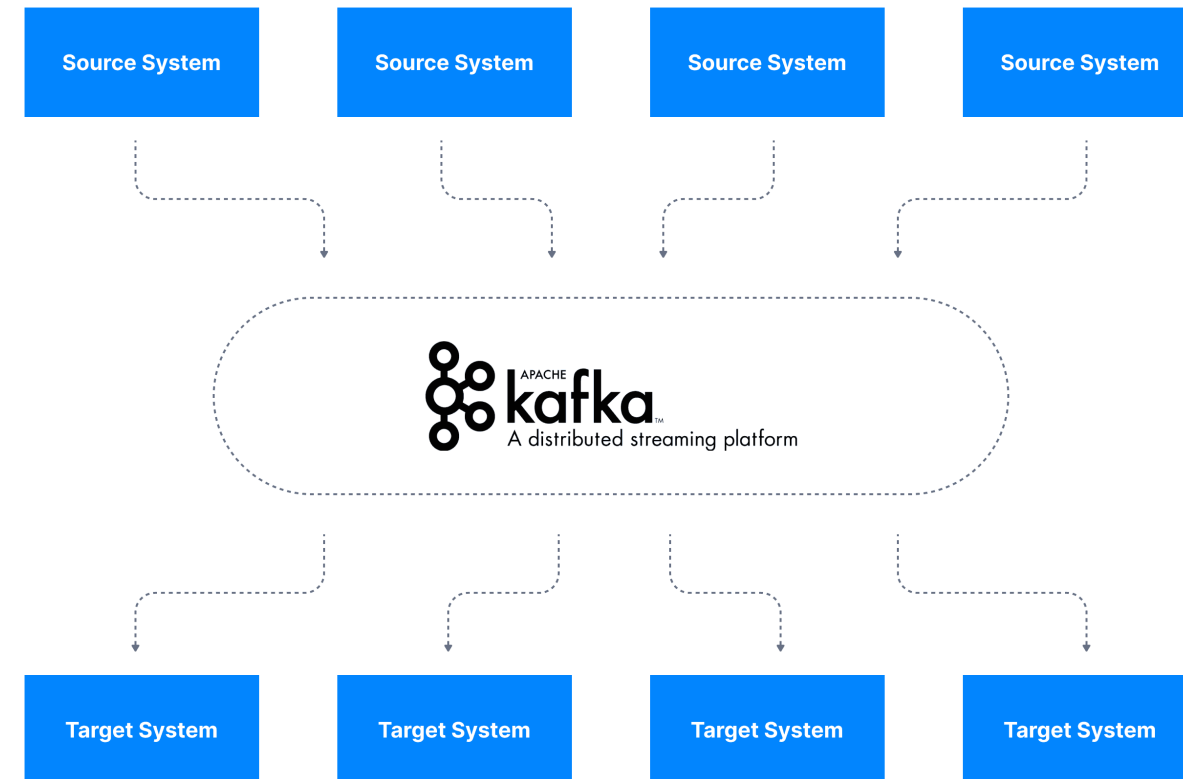


Izvor: <https://flink.apache.org/what-is-flink/flink-architecture/>



Apache Kafka

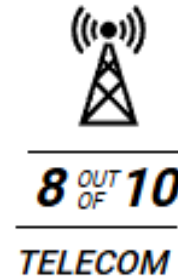
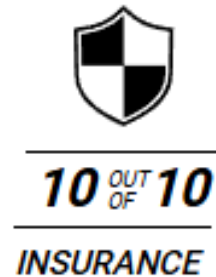
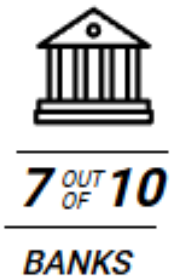
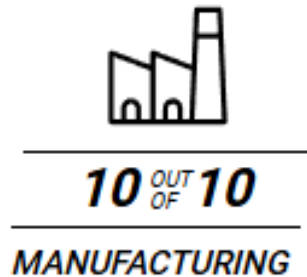
- Apache Kafka je **otvorena i raspodijeljena platforma** za obradu tokova podataka u stvarnom vremenu napisana u **Javi i Scali**
- Omogućuje **pouzdanu i brzu razmjenu velike količine podataka** među tim sustavima
- Odvaja izvorne i odredišne sustave te **olakšava integraciju podataka** (isti protokol, format, shema podataka)



Izvor: <https://learn.conduktor.io/kafka/what-is-apache-kafka>

Popularnost platforme Apache Kafka

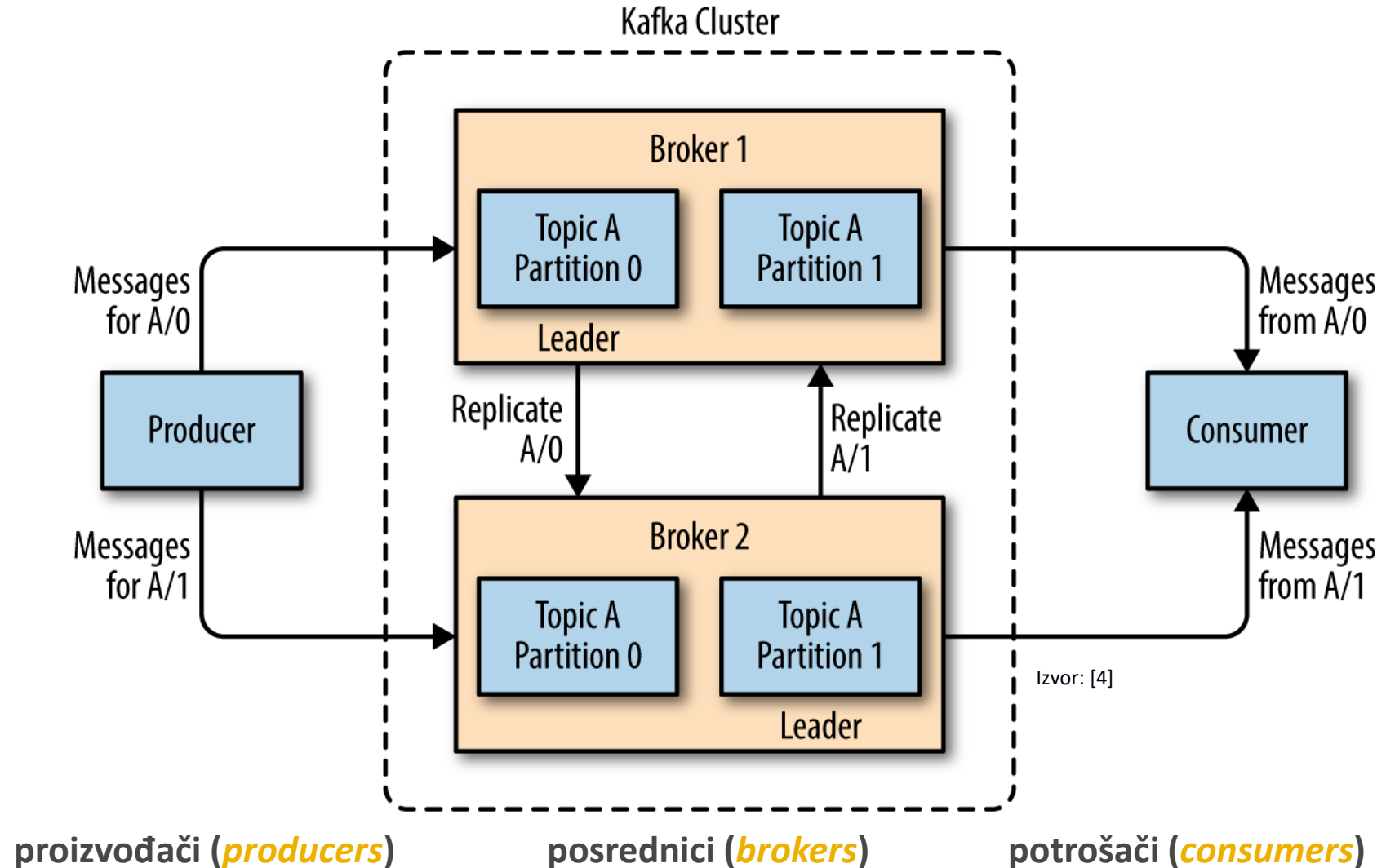
- Koristi ju 80% kompanija koje su među Fortune 100



- Razvio ju je LinkedIn i 2011. godine prepustio zajednici otvorenog koda
- LinkedIn navodi da ima više od **100 Kafka grozdova** (*cluster*) s više od **4.000 posrednika** (*broker*) na kojima se dnevno obradi više od **7 bilijuna (tisuća milijardi) poruka** (*message or record*) unutar više od **100.000 različitih tema** (*topic*) – preko **80 milijuna poruka u sekundi**

Arhitektura platforme Apache Kafka

- Osnovne Kafkine optimizacije
 1. Teme su **partitionirane** radi poboljšanja performansi obrade
 2. Particije tema su **replicirane** radi otpornosti na neispravnosti



Ključne Kafkine optimizacije

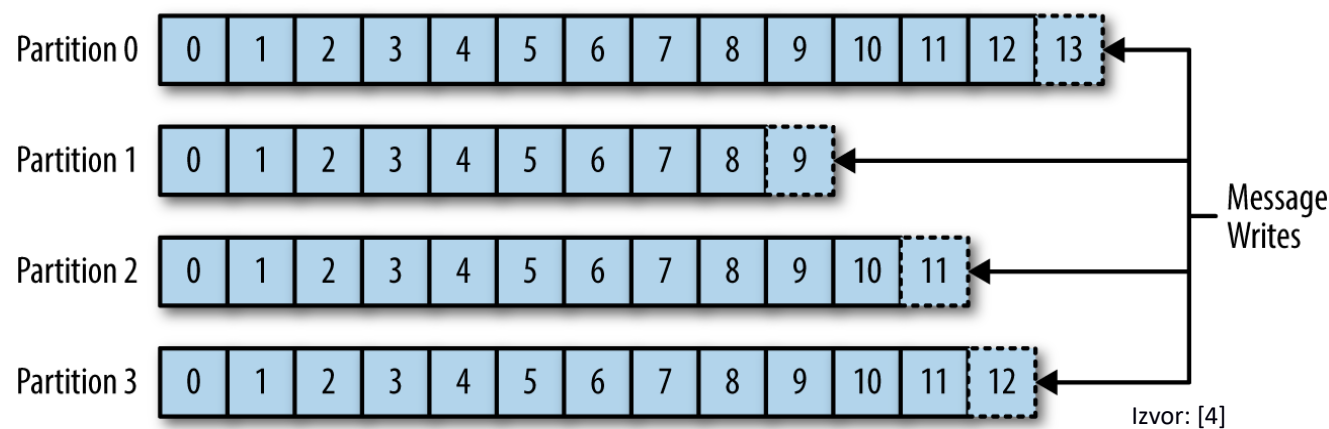
3. Kafka **zapisuje i čita podatke sekvencijalno na disku** (*Sequential I/O*)
 - smanjuje se vrijeme traženja na disku u usporedbi s nasumičnim pristupom
4. Mehanizam *Zero-Copy* omogućuje Kafki **izravan prijenos podataka između diska i mrežnog sučelja** bez prolaska kroz memoriju
 - smanjuje se korištenje procesora i ubrzava prijenos podataka
5. Kafka pohranjuje poruke u formatu **zapisnika u kojeg se samo dodaje** (*Append-Only Log*)
 - poruke se zapisuju na kraj zapisnika, a potrošači ih čitaju sekvencijalno pa su operacije pisanja i čitanja jako brze
6. Pohranjene **poruke su nepromjenjive** (*immutable*)
 - pojednostavljena kontrola konkurentnosti i poboljšavane performanse

Dodatne Kafkine Optimizacije

7. Kafka **grupira poruke zajedno** prije nego što ih pošalje ili zapiše na disk
 - Smanjena složenost mrežnih i diskovnih operacija spajanjem manjih operacija u veće
8. Kafka **podržava kompresiju poruka** u različitim formatima
 - smanjena veličinu poruka i količina podataka koji se prenose mrežom
9. Potrošači **sami prate svoj pomak** (*offset*)
 - Posrednici ne moraju održavati stanje za svakog potrošača
10. Kafka **koristi optimizirani binarni protokol za komunikaciju** između proizvođača (*producers*) i potrošača (*consumers*) te posrednika (*brokers*)
 - smanjen je *overhead* u odnosu na tekstualne protokole

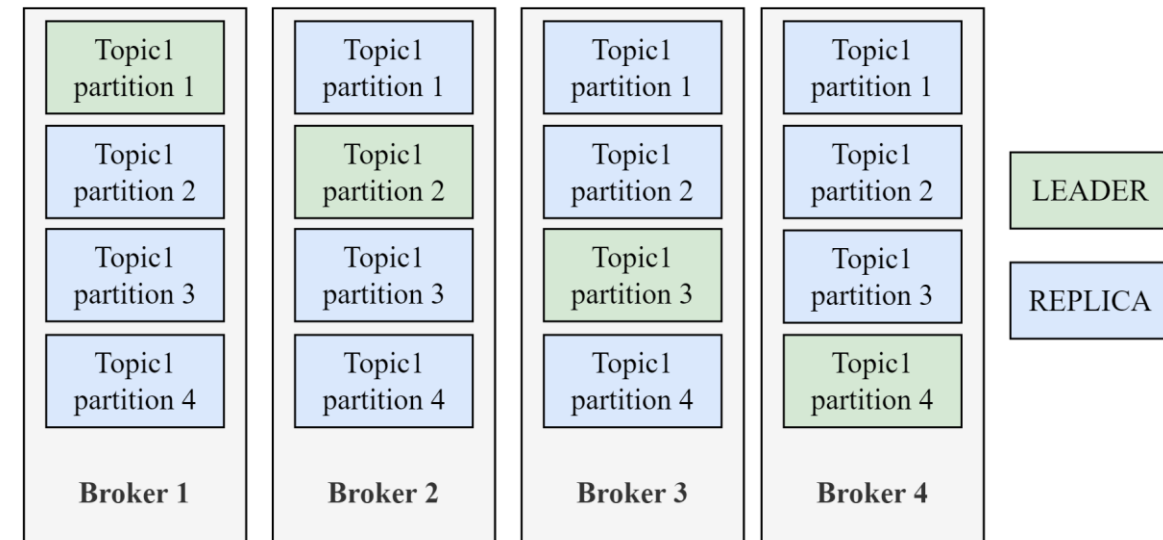
Partitioniranje tema

- Partitioniranje tema omogućava
 - čitanje i pisanje poruka na temi **u paraleli** te
 - **ravnomjernu raspodjelu opterećenja** među posrednicima
- **Poruka** se sastoji se od **ključa**, **vrijednosti** i **vremenske oznake**
- Tema kojoj pripada poruka se određuje prilikom objavljivanja
- Particija teme u koju će se zapisati poruka se određuje na osnovu ključa (tj. **hash vrijednosti ključa**)
- **Struktura podataka** koja se koristi za pohranu particije teme je **zapisnik** (*append-only log*)
- Poruke s **istim ključem** uvijek se uvijek pohranjuju u **istu particiju** teme
- Poruke unutar iste particije čitaju se **redoslijedom kojim su pohranjene**



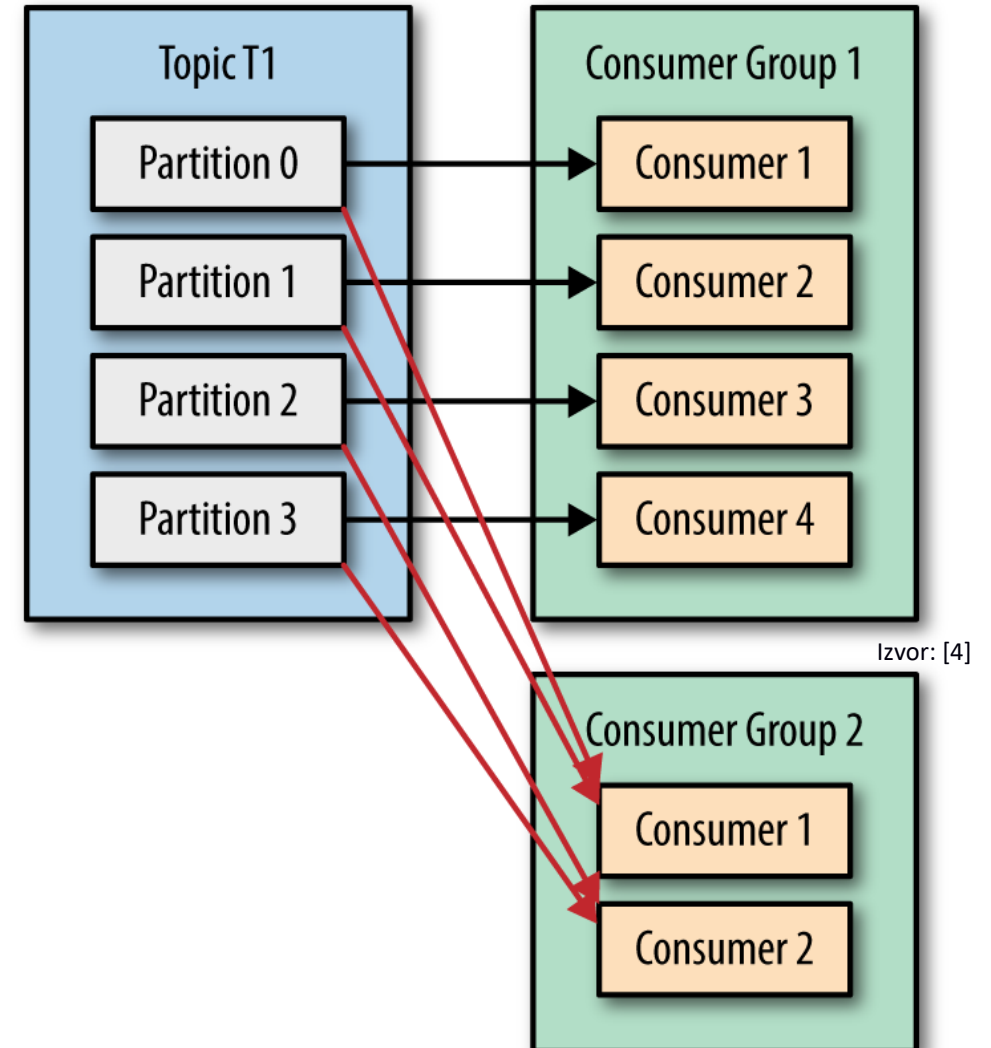
Replikacija na posrednicima

- **Replikacija particija** na više posrednika **osigurava dostupnost podataka** čak i ako neki posrednik prestane raditi
- Svaka particija ima jednog **vođu** (*partition leader*) koji je zadužen za **primanje i slanje podataka klijentima** za tu particiju
- Ostali posrednici služe kao **rezerve** i **preuzimaju ulogu vođe** ako je originalni vođa nedostupan
- Na ovaj način se osigurava visoka **pouzdanost, performanse** i **otpornost sustava na neispravnosti**



Grupe potrošača (*consumer groups*)

- Grupa potrošača je skup potrošača koji rade u paraleli i međusobno dijele posao
- Dva različita potrošača iz iste grupe ne mogu konzumirati istu particiju iste teme
- Jedan potrošač može konzumirati više particija iste teme
- Maksimalni paralelizam se postiže kada broj potrošača u grupi odgovara broju particija teme



Najvažnija programska sučelja u Kafki

- **Producer API** – omogućava aplikacijama slanje podataka u Kafkine teme
- **Consumer API** – omogućava aplikacijama čitanje podataka iz Kafkinih tema
- **Streams API** – omogućava obradu podataka (transformiranje, filtriranje, agregiranje i druge operacije) u stvarnom vremenu unutar Kafke
- **Connect API** – omogućava povezivanje vanjskih sustava (baze podataka, datotečni sustavi i druge aplikacije) s Kafkom putem gotovih konektora
- **Admin API** – omogućava administraciju i upravljanje Kafkinim grozdom klasterom
 - stvaranje, brisanje ili konfiguriranje tema
 - upravljati korisnicima pravima pristupa
 - nadzirati statusa grozda i particija

Pokretanje Kafkinog grozda

- Za testiranje aplikacije se koristi jedan testni posrednik
 - Pokretanje **korištenjem odgovarajućih skripti**
 - Pokretanje unutar Dockerovog kontejnera
- U produkciji je potrebno pokrenuti čitav Kafkin grozd
 - potrebno je pokrenuti nekoliko Kafkinih brokera na računalima u grozdu
 - računa u grozdu imaju instaliran operacijski sustav Linux

Pokretanje Kafkinog grozda korištenjem skripti

0. Potrebno je imati instaliran JDK 8+

- Microsoft Build of OpenJDK 25 se može dohvatiti sa sljedeće adrese:

<https://learn.microsoft.com/hr-hr/java/openjdk/download>

1. Dohvatiti binarnu verziju Kafke u obliku TGZ arhive sa sljedeće adrese:

<https://kafka.apache.org/downloads>

2. Raspakirati arhivu u neku mapu na disku npr. C:\KOMMRE2\

3. Preimenovati mapu iz C:\KOMMRE2\kafka_2.13-4.1.1 u
C:\KOMMRE2\kafka

Pokretanje Kafkinog grozda korištenjem skripti

4. U datoteci `C:\KOMMRE2\kafka\config\server.properties` promijeniti liniju **`log.dirs=/tmp/kraft-combined-logs`** u **`log.dirs=.\logs`**
5. Iz aplikacije **PowerShell** ući u preimenovnu mapu i pokrenuti sljedeće naredbe:
 - `$env:KAFKA_CLUSTER_ID="$(bin\windows\kafka-storage.bat random-uuid),,`
 - `$env:KAFKA_LOG4J_OPTS="-Dlog4j.configurationFile=./config/log4j2.yaml"`
 - `.\bin\windows\kafka-storage.bat format -s -t $env:KAFKA_CLUSTER_ID -c .\config\server.properties`
 - `.\bin\windows\kafka-server-start.bat .\config\server.properties`

Stvaranje Kafkinih tema iz komandne linije

- Otvoriti novi prozor u aplikaciji **PowerShell** te ući u preimenovnu mapu i pokrenuti sljedeće naredbe:
 - `$env:KAFKA_LOG4J_OPTS="-Dlog4j.configurationFile=./config/log4j2.yaml"`
 - `.\bin\windows\kafka-topics.bat --create --topic taxi_rides --bootstrap-server localhost:9092`
 - `.\bin\windows\kafka-topics.bat --describe --topic taxi_rides --bootstrap-server localhost:9092`
 - `.\bin\windows\kafka-topics.bat --alter --topic taxi_rides --partitions 2 --bootstrap-server localhost:9092`
 - `.\bin\windows\kafka-topics.bat --describe --topic taxi_rides --bootstrap-server localhost:9092`
 - `.\bin\windows\kafka-topics.bat --create --topic aggregated_tips --partitions 2 --bootstrap-server localhost:9092`
 - `.\bin\windows\kafka-topics.bat --describe --topic aggregated_tips --bootstrap-server localhost:9092`

Programski primjer Kafkinog proizvođača

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092"); // kafka broker(s)
props.put("key.serializer", "org.apache.kafka.common.serialization.LongSerializer");
props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");

long counter = 0;
try (KafkaProducer<Long, String> producer = new KafkaProducer<>(props)) {
    for (String line : Files.readAllLines(Path.of(CSV_FILE))) {
        ProducerRecord<Long, String> record = new ProducerRecord<>(TOPIC, counter++, line);
        try {
            RecordMetadata metadata = producer.send(record).get();
            //do something with the metadata
        } catch (ExecutionException | InterruptedException e) {
        }
    }
} catch (IOException e) {
}
```

Programski primjer Kafkinog potrošača

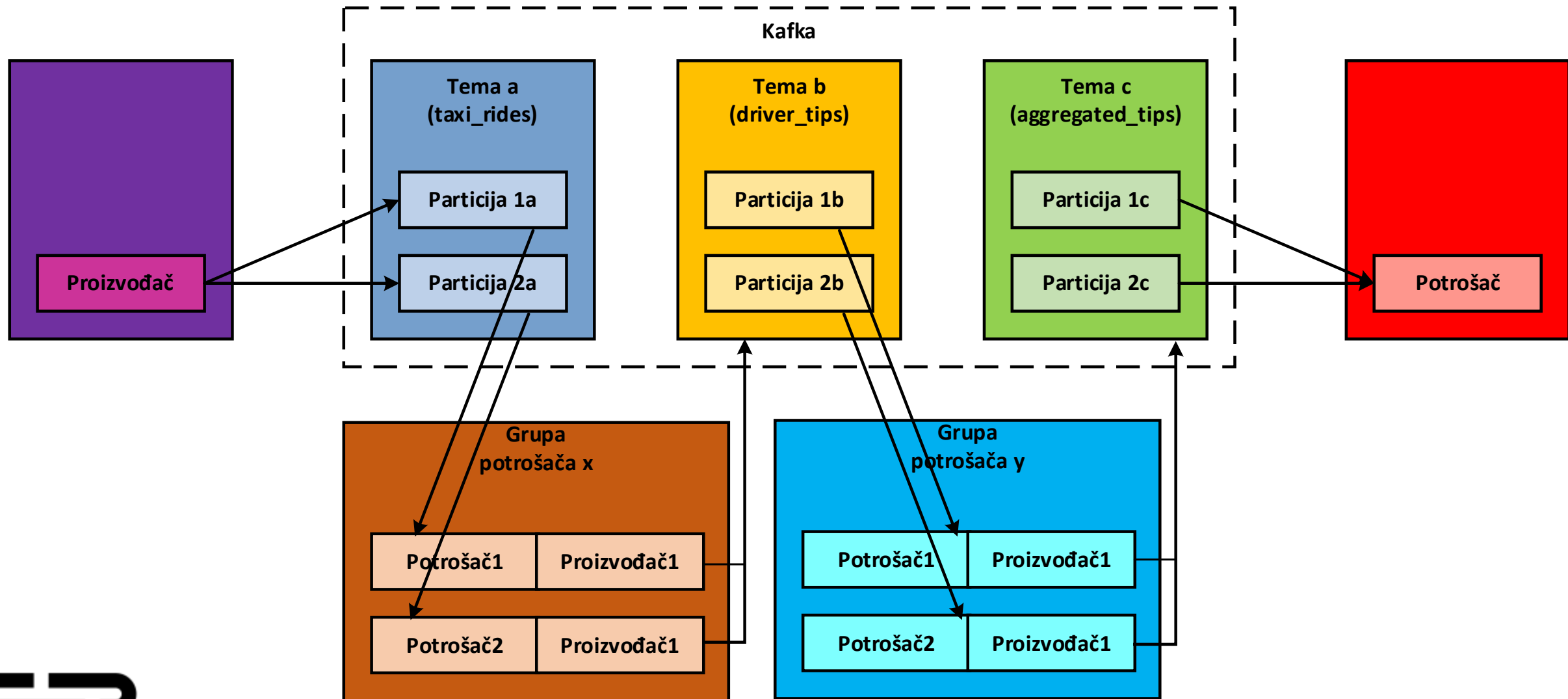
```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("group.id", CONSUMER_GROUP_ID); //consumer group
props.put("key.deserializer", "org.apache.kafka.common.serialization.LongDeserializer");
props.put("value.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
props.put("auto.offset.reset", "earliest"); //offset starts from the beginning

try (KafkaConsumer<Long, String> consumer = new KafkaConsumer<>(props)) {
    consumer.subscribe(List.of(TOPIC));
    while (true) {
        ConsumerRecords<Long, String> records = consumer.poll(Duration.ofSeconds(1));
        for (ConsumerRecord<Long, String> record : records) {
            //do something with the record
        }
    }
} catch (Exception e) {
}
```

Problem: Kako zbrojiti napojnice po taksistima?

- Svaki objavljena poruka je formata: medallion, **hack_license**, pickup_datetime, dropoff_datetime, trip_time_in_secs, trip_distance, pickup_longitude, pickup_latitude, dropoff_longitude, dropoff_latitude, payment_type, fare_amount, surcharge, mta_tax, **tip_amount**, tolls_amount, total_amount
- Potrošači ne mogu direktno zbrajati napojnice (tip_amount) u paraleli jer se poruke s istim taksistom (hack_license) pojavljuju u različitim particijama ulaznih podataka
 - Potrošači bi trebali parsirati ulazne podatke i ujedno biti objavljiivači parova ključ-vrijednost <hack_license,tip_amount> na novu temu
 - Potrošači te nove teme bi mogli raditi u paraleli jer bi podaci o istom taksistu završili u istoj temi jer ključ određuje particiju teme kojoj poruka pripada

Komplicirano rješenje – obrada u slijedu



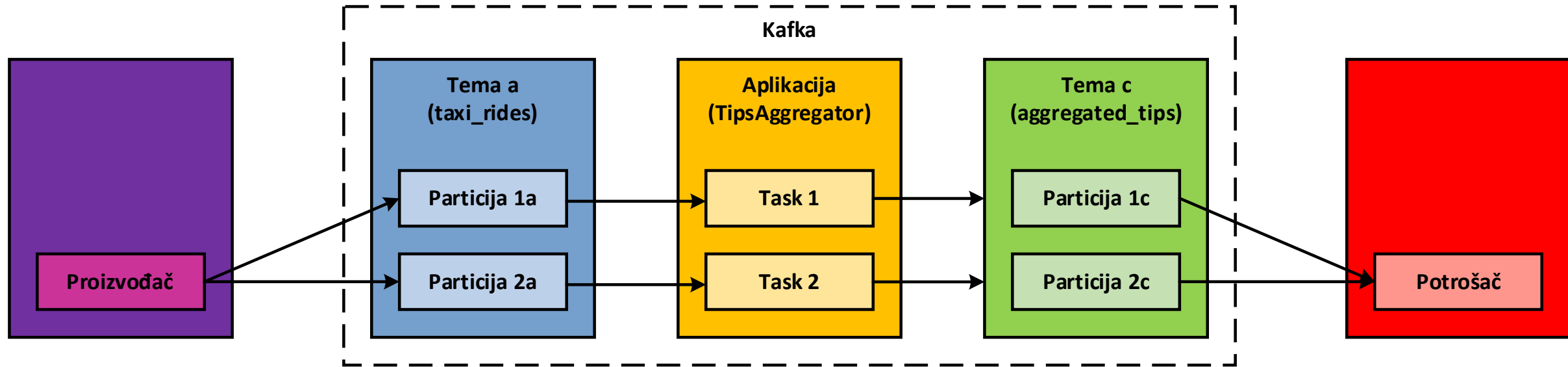
Kafka Streams API

- Tri inačice:
 - **Kafka Streams DSL** (visoka razina apstrakcije),
 - Kafka Processor API (niska razina apstrakcije),
 - KSQL (temelji se na SQL-u)
- Koristi Kafkine teme kao ulaz i izlaz te omogućava kontinuiranu obradu toka podataka
- Razlika u odnosu na Producer API i Consumer API
 - ne zahtijeva ručno upravljanje čitanjem i pisanjem podataka
 - automatizira te zadatke te pojednostavljuje razvoj
 - nudi ugrađene operacije poput mapiranja, filtriranja, grupiranja i agregiranja
 - automatski paralelizira i skalira obradu
 - integrira obradu podataka bez potrebe za dodatnim platformama (kao što su Apache Spark i Apache Flink)

Osnovne klase Kafka Streams DSL API-ja

- Klasa `StreamsBuilder`: koristi za definiranje **topologije aplikacije**
 - logički prikaz procesa obrade podataka
 - definira kako podaci prolaze kroz aplikaciju, kako se transformiraju i gdje se šalju
- Klasa `KStream`: predstavlja **tok zapisa** (neprekidni tok podataka)
- `KTable`: predstavlja **tok promjena** koji održava pregled najnovije vrijednosti za svaki ključ

Poboljšano rješenje – obrada toka podataka



Programski primjer Kafkine aplikacije

```
Properties props = new Properties();
props.put("application.id", "tips-aggregator-9");
props.put("bootstrap.servers", "localhost:9092");
props.put("default.key.serde", Serdes.String().getClass().getName());
props.put("default.value.serde", Serdes.Double().getClass().getName());

StreamsBuilder builder = new StreamsBuilder();
KStream<Long, String> inStream = builder.stream(INPUT_TOPIC, Consumed.with(Serdes.Long(),
Serdes.String()));

inStream
    .map((key, value) -> new KeyValue<>(value.split(",")[1], Double.valueOf(value.split(",")[14])))
    .groupByKey() //KGroupedStream<String, Double>
    .reduce((total, tip) -> total + tip) //KTable<String, Double>
    .toStream() //KStream<String, Double>
    .to(OUTPUT_TOPIC);

new KafkaStreams(builder.build(), props).start();
```

Lista transformacija Kafka Streams DSL API-ja

- <https://docs.confluent.io/platform/current/streams/developer-guide/dsl-api.html>

Literatura

- [1] <https://www.pcmag.com/encyclopedia/term/big-data>
- [2] Prajapati, V. (2013). Big data analytics with R and Hadoop (pp. 112-117). Birmingham: PACKT publishing.
- [3] Golab, Lukasz, and M. Tamer Özsu. "Issues in data stream management." ACM Sigmod Record 32.2 (2003): 5-14.
- [4] Narkhede N., Shapira G., Palino T. Kafka: The Definitive Guide: Real-Time Data And Stream Processing At Scale. O'Reilly Media, 2017.